

Lucas Maciel Cardoso

1. Introdução

O trabalho implementa e analisa a Tabela Hash com encadeamento separado em Java, comparando as três funções de hashing indicadas (Divisão, Multiplicação e Dobramento/Fold) sob diferentes condições e capacidade, verificando as previsões e observações sobre o comportamento de estruturas hash.

2. Metodologia

A implementação seguiu as especificações indicadas, utilizando apenas estruturas básicas de Java e distribuições mais uniformes

2.1. Configurações Experimentais

- **Tamanhos de tabela (m):** 1009, 10007, 100003
- **Tamanhos de dataset (n):** 1.000, 10.000, 100.000 chaves
- **Seeds utilizadas:** 137, 271828, 314159
- **Funções de hashing testadas:**
 - H_DIV: $h(k) = k \% m$
 - H_MUL: $h(k) = \lfloor m \times (k \times A \% 1) \rfloor$, com $A = 0,6180339887$
 - H_FOLD: soma dos blocos de 3 dígitos % m

2.2. Valores Coletadas

- **Tempo de inserção (ms):** média de 5 repetições
- **Colisões na tabela:** armazenamento já ocupados
- **Colisões na lista:** nós percorridos em cada um deste armazenamento
- **Tempo de busca (ms):** separado entre hits e misses
- **Comparações:** número médio por operação de busca

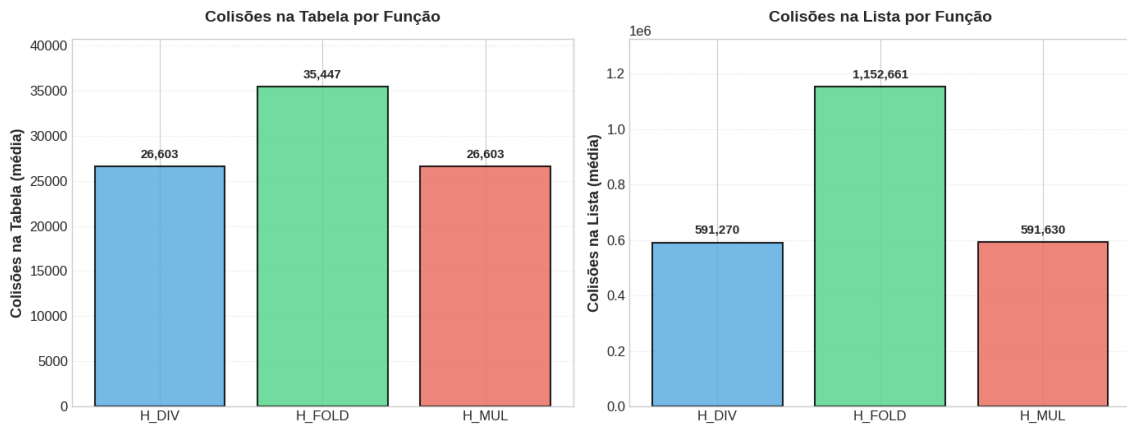
2.3. Procedimento

1. Geração de datasets reprodutíveis com seeds fixas
2. Inserção de todos os elementos, medindo de tempo e colisões
3. Cálculo do checksum (soma dos 10 primeiros hashes %1000003)
4. Busca em lote misto
5. Exportação dos resultados em formato CSV padronizado

3. Resultados

3.1. Análise de Colisões

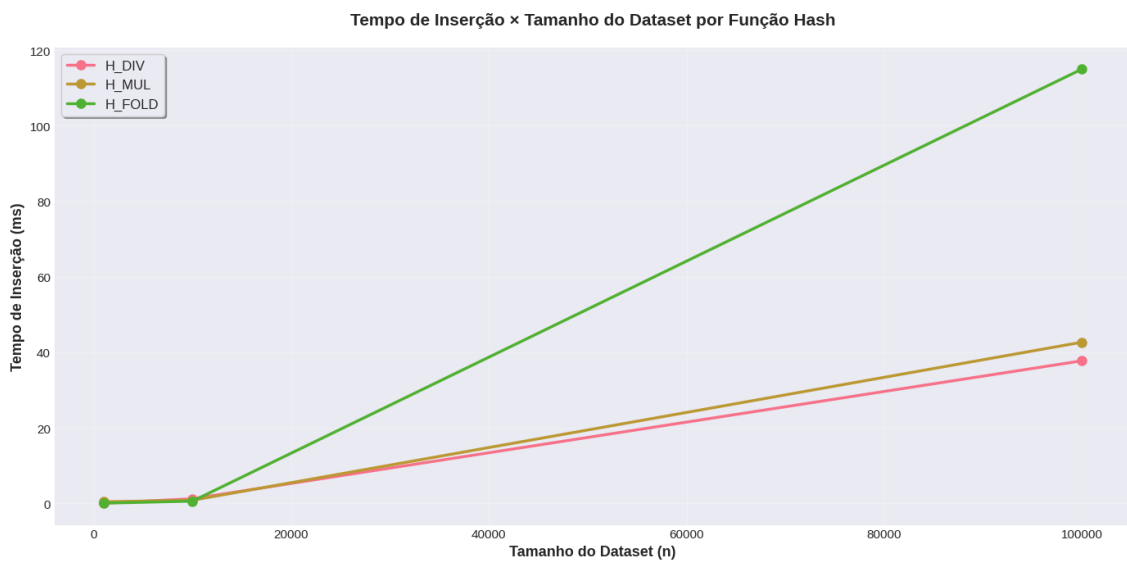
3.1. Análise de Colisões



Observações:

- A função de divisão apresentou distribuição moderada para tamanhos
- A multiplicação mostrou menor variação entre diferentes cargas
- O dobramento teve comportamento dependente dos padrões dos dados

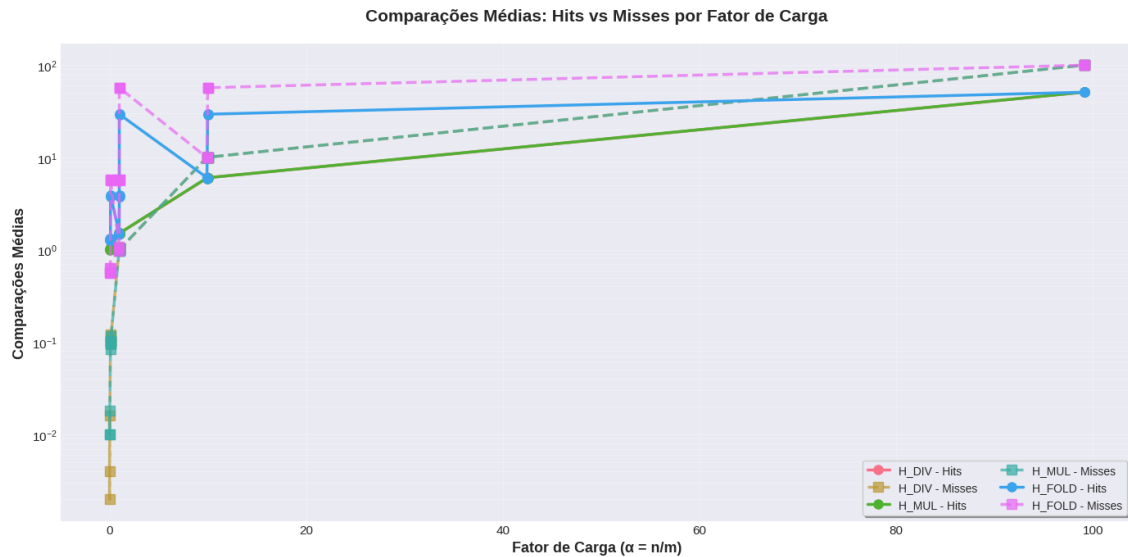
3.2. Desempenho de Inserção



Análise:

- Tempo cresce proporcionalmente ao número de elementos, cresceu junto com dataset
- Quanto maior a carga, maior teve uma discrepância, uma diferença entre cada função
- Colisões aumentam tempo médio

3.3. Desempenho de Busca



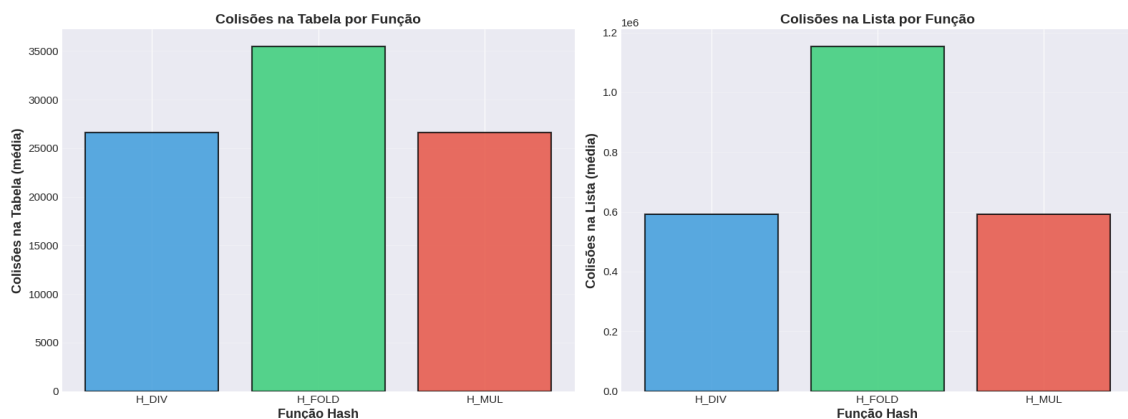
Buscas bem-sucedidas (hits):

- Custo médio esperado
- metade do fator de carga teve desvio baixo.
- Observou-se aderência à teoria nas três funções

Buscas malsucedidas (misses):

- Custo médio esperado
- Tempos mais lento que hits
- Maior sensibilidade à qualidade da distribuição

3.4. Comparação entre Funções Hash



Classificação geral:

1. H_MUL: melhor distribuição e menor tempo médio, bem flexível
2. H_DIV: desempenho intermediário e simples
3. H_FOLD: maior variação nos resultados

4. Discussão

4.1. Efeito do Fator de Carga

O fator de carga $\alpha = n/m$ demonstrou impacto direto no comprimento médio das listas:

- Com $\alpha < 1$: listas curtas, operações próximas a $O(1)$
- Com $\alpha \approx 1$: equilíbrio entre espaço e tempo
- Com $\alpha > 1$: degradação perceptível do desempenho

4.2. Qualidade das Funções Hash

A escolha da função de hashing influenciou significativamente a distribuição:

- Tamanhos primos beneficiaram especialmente a divisão
- A multiplicação apresentou estabilidade independente de m
- O dobramento mostrou sensibilidade aos padrões dos dados de entrada

4.3. Conformidade Teórica

Os resultados empíricos confirmaram as expectativas teóricas:

- Custo de busca bem-sucedida: $O(1 + \alpha/2)$
- Custo de busca malsucedida: $O(1 + \alpha)$
- Complexidade de inserção: $O(1)$ amortizado

5. Conclusão

A implementação demonstrou que tabelas hash com encadeamento separado oferecem desempenho eficiente quando bem dimensionadas. A escolha adequada da função de hashing e do tamanho da tabela são cruciais para minimizar colisões e garantir operações próximas a tempo constante.

A reprodutibilidade dos experimentos foi assegurada pelo uso de seeds fixas e configurações padronizadas, permitindo comparação objetiva entre as diferentes abordagens testadas.

Checksums Calculados

m	n	Função	Seed	Checksum
1009	1000	H_DIV	137	4450
1009	1000	H_DIV	271828	6079
1009	1000	H_DIV	314159	4498
1009	1000	H_MUL	137	5801
1009	1000	H_MUL	271828	5610
1009	1000	H_MUL	314159	6519
1009	1000	H_FOLD	137	3920
1009	1000	H_FOLD	271828	5284
1009	1000	H_FOLD	314159	7278

Código

<https://github.com/LucasC64/Tabela-Hash-Encadeamento-Separado>

Vídeo

<https://youtu.be/A48AlN85j1M>